Hardware — Learning Content V1.1

*Part of the module **Hardware, Networks & Behaviorist AI**. The Networks and Behaviorist AI units are developed on their own branches/folders; this file covers the Hardware unit only.*

How to use this file

The Hardware unit answers two beginner questions:

1. **What is inside a computer, and how does it follow instructions?** (Part A — matches Lessons 1–2 in study-overview.md)
2. **How does a computer-based system sense and affect the physical world?** (Part B — a worked project: an automatic light indicator)

Part A explains the computer as a general idea. Part B uses a small computer (a microcontroller) to build something real. Section A6 is the bridge between the two. Part C goes further: serial buses (I^2C, SPI, UART), sending data to another device, capacitors, reading a real datasheet, and e-waste.

Two companion files support Part B:

- beginner-lab-skills.md — breadboards, multimeters, resistor colour codes, uploading code, schematic symbols, digital inputs, glossary and Questions H–L. Read it before the first lab session.
- arduino/ — the B11 sketches as ready-to-open Arduino files, plus calibration, PWM-dimming and push-button extensions.

A unifying example for the whole module is a **smart-room system**: a sensor measures brightness, a controller reads the measurement, a network may share the reading, an agent or control rule decides what should happen, and an output device switches a light. This file covers the sensor, controller and output parts.

Prerequisites

Beginners do not need previous electronics or programming experience. They should be prepared to learn these ideas as they go:

- Basic arithmetic, fractions, and rearranging a simple equation.
- The difference between physical hardware and software instructions.
- Basic programming ideas such as variables, conditions (`if` statements), and repetition (`loop`).

Every technical term is defined the first time it is used.

Intended learning outcomes

By the end of the Hardware unit, a beginner should be able to:

1. **Identify** input devices, the CPU, RAM, storage and output devices, and describe their roles.
2. **Explain** bits and bytes with a small example and **trace** a simplified fetch–decode–execute cycle.
3. **Describe** how a microcontroller relates to a full computer, and when each is a better choice.
4. **Explain and calculate** simple circuits using Ohm's law, series/parallel rules and the voltage-divider equation.

5. **Select and justify** components using voltage, current, power, interface, performance, environment, cost and availability.
6. **Trace** the path from a physical measurement through a sensor and controller to an actuator.
7. **Explain** why motors, relays and other heavy loads need driver circuits, and how PWM controls brightness or speed.
8. **Evaluate** a small hardware design for compatibility, power budget, safety and testing.

Unit structure and development status

Status meanings:

- **Outline only:** the plan exists, but teaching material still needs to be written.
- **Drafted:** a first version exists; it has not yet passed peer or instructor review.
- **Checked:** another person or a documented test has checked the material. Use this label only after that check has happened.
- **Needs revision:** a known gap or uncertainty remains.

Section	Topics	Learners can…	Example / activity	Status
A1–A2	Computer components; RAM vs storage	Name each part's job	Follow a photo from storage to screen	**Drafted — needs peer review**
A3–A4	Bits, bytes, logic gates	Convert 4-bit binary; read a truth table	Bit cards 0–15	**Drafted — needs peer review**
A5	Fetch–decode–execute cycle	Trace a 3-instruction program	Paper CPU trace (Question G)	**Drafted — needs peer review**
A6	Microcontroller vs computer vs single-board computer	Choose a platform for a task	Arduino Uno vs Raspberry Pi	**Drafted — needs peer review**
B1–B4	Electrical quantities, Ohm's law, series/parallel, safety	Do a one-step circuit calculation	Series/parallel worked example	**Drafted — needs technical review**
B5	Component selection, datasheets, BOM	Justify a part choice	BOM for the light indicator	**Drafted — needs technical review**
B6–B7	Voltage divider, ADC, resolution/accuracy, smoothing, calibration, hysteresis	Explain how light becomes a number	LDR on A0	**Drafted — needs technical review**
B8–B10	LED output, actuators, transistor/MOSFET drivers,	Explain why a motor needs a	Transistor base-resistor calculation	**Drafted — needs**

Section	Topics	Learners can…	Example / activity	Status
	PWM, servos, power budget	driver		**circuit/peer review**
B11–B15	Code, system diagram, activity, answers, troubleshooting	Build, test and debug the circuit	Questions A–G	**Drafted — not yet tested on real hardware**
A3, A5, A6 additions	Hexadecimal; buses, MAR/MDR, clock; von Neumann vs Harvard	Convert hex; trace a fetch on the buses	Paper CPU re-traced with buses; Questions M, N, R	**Drafted — needs peer review (several secondary sources)**
C1–C6	UART, I^2C, SPI; CSV over serial; capacitors; datasheet walkthrough; e-waste and RoHS	Choose a bus; read a datasheet; explain safe disposal	BH1750 vs LDR; GL5528/WP7113ID margin table; Questions M–R	**Drafted — needs technical review (several secondary sources)**
L1–L8 (beginner-lab-skills.md)	Breadboard, multimeter, resistor codes, Arduino IDE, schematic symbols, digital inputs, glossary	Build and measure a circuit safely	Measure the LDR; Questions H–L	**Drafted — needs technical review**

Where this fits in the full module

Rows 2–6 are developed in this file; rows 7–10 belong to the Networks and Behaviorist AI units.

Order	Topic and subtopics	What learners should be able to do	Planned example or activity	Current status
1	Module overview: hardware, networks, and agent behaviour	Describe how the parts of a connected system work together	Trace a smart-room system from sensor reading to light output	**Drafted; needs team review**
2	Hardware foundations: voltage, current, resistance, power, series and parallel circuits, basic safety	Define the main electrical quantities and use Ohm's law	Calculate current through a resistor	**Drafted; needs technical review** (B2–B4)
3	Physical component selection: microcontrollers, single-board computers, sensors, resistors, drivers, power supplies, datasheets, trade-offs	Choose components and justify the choices	Choose a controller and light sensor for an automatic lamp	**Drafted; needs technical review** (A6, B5)

Order	Topic and subtopics	What learners should be able to do	Planned example or activity	Current status
4	Circuit design: schematics, voltage dividers, current-limiting resistors, logic levels, power budget, grounding, protection	Explain and check a simple circuit	Calculate a voltage-divider output and an LED resistor	**Drafted; needs circuit review** (B5–B6, B8, B10)
5	Sensor integration: analog and digital sensors, ADC, accuracy, resolution, signal conditioning, sampling, calibration	Explain how a measurement becomes a value that software can use	Read an LDR through an analog input	**Drafted; needs technical review** (B6–B7)
6	Actuator integration: LEDs, buzzers, relays, motors, GPIO limits, transistor/MOSFET drivers, PWM, protection	Explain when a load needs a driver circuit	Compare an LED output with a motor output	**Drafted; needs circuit review** (B8–B9)
7	Network foundations: links, packets, addresses, Ethernet, Wi-Fi, switches, routers, access points, latency, reliability	Describe a basic device-to-device communication path	Draw how a sensor sends a reading to a computer	**Outline only; coordinate with Networks contributor**
8	Networked and IoT systems: device communication, data formats, connectivity, privacy, security	Identify what a connected device needs beyond a sensor and controller	Plan how a room sensor could share readings with another device	**Outline only; content still needed**
9	Behaviorist AI or behaviour-based agents: inputs, state or feedback, rules, action selection	Explain the approach used in this course and trace an input-to-action example	Describe how a robot might respond to an obstacle	**Outline only; clarify the course meaning of "Behaviorist AI"**
10	Integration and evaluation: hardware, networks, agent behaviour, reliability, testing, trade-offs, documentation	Explain how the pieces work together and identify likely failure points	Review a connected smart-room design	**Outline only; depends on agreement across sections**
11	Glossary, questions, and references	Use terms consistently and check understanding	Answer short questions with explanations	**Hardware terms drafted; full-module glossary incomplete**

Known gaps and uncertainties

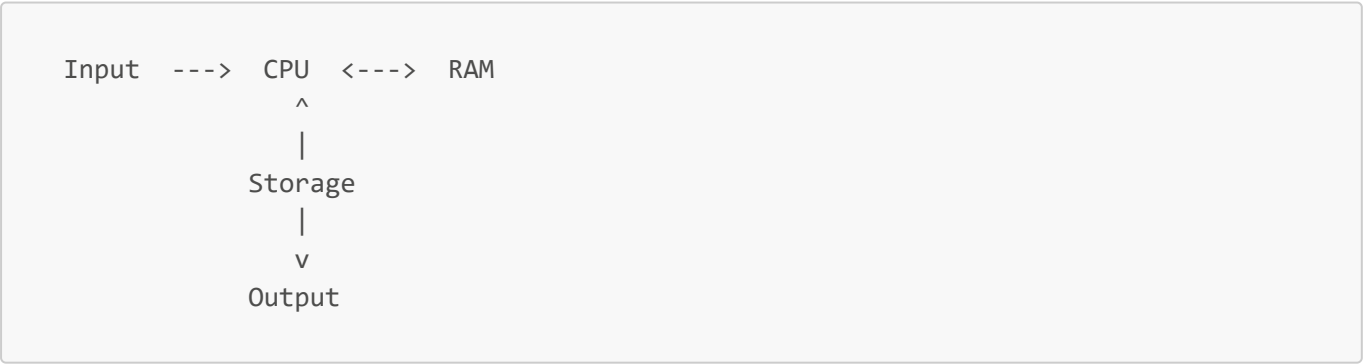
- Circuit values are **illustrative**. A real build must use the chosen parts' datasheets and the exact board's specifications.
- The examples assume a classic **5 V Arduino Uno R3**. Other boards differ (see B5 *Logic levels*).
- Some exact figures could not be read from the primary Microchip ATmega328P datasheet during research and come from secondary sources; these are marked *(secondary source)*.
- The circuit has **not yet been built and tested** by the team. Do not mark it *Checked* until it has.

Part A — Inside a computer

A1. The main parts and their jobs

A **computer** is a machine that takes in data, processes it by following instructions, and produces results. Every computer — laptop, phone, games console, or the small board in Part B — has the same basic jobs:

Part	Job	Everyday example
Input device	Brings data in from the outside world	Keyboard, mouse, camera, microphone, light sensor
CPU (central processing unit)	Follows instructions: does calculations and makes decisions	The "brain" chip in a laptop
RAM (random-access memory)	Holds the programs and data being used *right now*	Your open documents and browser tabs
Storage	Keeps programs and files when the power is off	SSD, hard disk, USB drive, memory card
Output device	Sends results back to the outside world	Screen, speaker, printer, LED, motor



Demonstration — opening a photo: the photo file sits in *storage*. When you open it, a copy is loaded into *RAM*. The *CPU* runs the photo-viewer program's instructions to decode the image, and the result is sent to the *screen* (output). The mouse click that started it all was an *input*.

A2. RAM vs storage — a common confusion

- **RAM is volatile:** it needs power to keep its contents. When the power goes off, everything in RAM is lost. RAM is fast, so the CPU works from it.
- **Storage is non-volatile:** it keeps data without power. It is slower than RAM but holds much more.

So unsaved work (only in RAM) is lost in a power cut; a saved file (in storage) is not. A laptop with "16 GB RAM, 512 GB SSD" has 16 GB of working space and 512 GB of long-term space — the two numbers describe different jobs. [Corsair, volatile vs non-volatile memory]

A3. Bits and bytes

A **bit** (binary digit) is the smallest unit of data: it is either **0** or **1**. Hardware stores a bit as one of two physical states, such as low or high voltage.

A **byte** is a group of **8 bits**.

Groups of bits can represent numbers. Each position is worth double the one to its right:

Position value	8	4	2	1
Bits	1	0	1	1

$1011 = 8 + 0 + 2 + 1 = 11$

- 4 bits can show $2^4 = 16$ values: 0 to 15.
- 8 bits (one byte) can show $2^8 = 256$ values: 0 to 255.

Misconception: binary is a way of *representing* information. It does not mean the computer *understands* the information as a person does.

Units note: officially, 1 kB (kilobyte) = 1000 bytes and 1 KiB (kibibyte) = 1024 bytes. Many product pages use "KB" loosely for 1024. Also watch the letter case: **b** = bit, **B** = byte. [NIST, prefixes for binary multiples]

Hexadecimal: a shorthand for binary

Hexadecimal (hex) is base 16. It uses the digits 0–9 and then the letters A–F for the values 10–15. Its value to people is that **one hex digit stands for exactly 4 bits** (a *nibble*), so a byte is always **two** hex digits (0x00 to 0xFF = 0 to 255). Hex is shorter than binary and easier to copy without mistakes. The computer still stores binary — hex is only a way of *writing* it. [Save My Exams, hexadecimal (secondary source)]

Denary	0–9	10	11	12	13	14	15
Hex	0–9	A	B	C	D	E	F
Binary	0000–1001	1010	1011	1100	1101	1110	1111

Converting: split the binary into groups of 4 from the right, and convert each group.

- $1011\ 0110 \rightarrow$ B, 6 $\rightarrow$ 0xB6 $= 11 \times 16 + 6 = 182$
- $200 = 12 \times 16 + 8 \rightarrow$ 0xC8 $= 1100\ 1000$
- 0x23 $= 0010\ 0011 = 2 \times 16 + 3 = 35$ — **not** twenty-three.

In Arduino code a hex number starts with `0x` (for example `0x7B`); upper- or lower-case letters both work. Binary numbers start with `0b` (older sketches use `B`). A number with no prefix is denary. [Arduino, integer constants]

Where learners will meet hex: I²C device addresses (C1 — a BH1750 light sensor is at `0x23`), memory addresses (a 16-bit address bus spans 0x0000–0xFFFF, see A5), web colours (`#66FF33` = 24 bits), MAC addresses (12 hex digits = 48 bits — useful in the Networks unit) and error codes. [Save My Exams, uses of hexadecimal (secondary source)]

Misconception: "A is a letter, so it can't be a number." In hex, A is simply the single digit for ten.

A4. Logic gates

A **logic gate** is a tiny circuit that takes one or more bits in and gives one bit out according to a fixed rule. CPUs are built from millions or billions of them. A **truth table** lists every input combination and the output.

A	B	A AND B	A OR B	NOT A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

- **AND** outputs 1 only if *both* inputs are 1. ("The door opens if you have a key AND you know the code.")
- **OR** outputs 1 if *at least one* input is 1.
- **NOT** flips its single input.

A5. How the CPU follows instructions: fetch–decode–execute

A **program** is a list of instructions stored in memory, alongside the data it uses. The CPU repeats the same three-step cycle for every instruction, billions of times a second in a laptop.

Key parts inside the CPU:

- **Program counter (PC):** holds the memory address of the *next* instruction.
- **Instruction register (IR, sometimes CIR):** holds the instruction currently being worked on.
- **Control unit (CU):** decodes the instruction and sends signals to the other parts.
- **ALU (arithmetic logic unit):** does the arithmetic (add, subtract) and logic (AND, OR, compare).
- **Accumulator (ACC):** a small working register that holds a result.

The cycle:

1. **Fetch:** the address in the PC is sent to memory; the instruction at that address is copied into the IR. The PC is then increased so it points to the next instruction.
2. **Decode:** the control unit works out what the instruction means — the *operation* (for example ADD) and the *operand* (what to use, for example memory address 5).
3. **Execute:** the control unit makes it happen — for example the ALU adds a number to the accumulator. A "jump" instruction can change the PC, which is how programs repeat and make decisions.

Some syllabuses list a fourth step, **store** (writing a result back), which this guide treats as part of *execute*. [Save My Exams; Computer Science Wiki]

Worked example: a paper CPU

This is a **made-up teaching machine**, not a real processor, but the idea is the same.

Address	Contents	Meaning
0	`LOAD 4`	Copy the value at address 4 into ACC
1	`ADD 5`	Add the value at address 5 to ACC
2	`STORE 6`	Copy ACC into address 6
3	`HALT`	Stop
4	`7`	data
5	`5`	data
6	`0`	data (result goes here)

Cycle	PC at fetch	IR after fetch	Execute	ACC after	Address 6 after
1	0	`LOAD 4`	ACC ← 7	7	0
2	1	`ADD 5`	ACC ← 7 + 5	12	0
3	2	`STORE 6`	address 6 ← 12	12	12
4	3	`HALT`	stop	12	12

Notice that **instructions and data sit in the same memory** — only the PC decides which addresses are treated as instructions.

Classroom activity: one learner is the CPU (holding PC, IR and ACC cards), one is memory (holding the address cards). The CPU must ask memory for each address — it cannot "see" ahead.

How the CPU talks to memory: buses, MAR and MDR

The trace above says the address is "sent to memory". It travels on **buses** — groups of parallel wires that connect the CPU to memory and input/output.

Bus	Carries	Direction
Address bus	The memory *location* to read or write	One way: CPU → memory / I/O
Data bus	The data or instruction itself	Two way
Control bus	Signals such as *read*, *write*, *clock*, *interrupt*, *reset*	Two way (each individual line has its own direction)

[Save My Exams, von Neumann architecture (secondary source); Toshiba, types of bus lines]

Width matters. An address bus with n wires can name 2^n locations — the same 2^n rule as A3. A 16-bit address bus reaches $2^{16} = 65\,536$ locations (0x0000–0xFFFF); each extra wire doubles that. [University of Cambridge Engineering, microprocessor notes]

Two more registers sit at the CPU's edge, where it meets the buses:

- **Memory address register (MAR):** holds the address about to be read or written. Its contents go out on the address bus.
- **Memory data register (MDR):** holds the data or instruction just read from memory, or about to be written to it. It connects to the data bus.

The fetch step in full: the PC is copied into the MAR → the address goes out on the address bus → the control unit sends *read* on the control bus → the instruction comes back on the data bus into the MDR → it is copied into the IR (CIR) → the PC is increased. [Seneca, Cambridge IGCSE fetch–decode–execute (secondary source); Save My Exams (secondary source)]

The paper CPU, re-traced with buses:

Instruction	What goes in MAR (address bus)	Control signal	What travels on the data bus
Fetch `LOAD 4`	0	read	`LOAD 4` → MDR → IR
Execute `LOAD 4`	4	read	7 → MDR → ACC
Execute `ADD 5`	5	read	5 → MDR → ALU adds to ACC
Execute `STORE 6`	6	**write**	12 from ACC → MDR → memory

The `STORE` row is why the data bus must be two-way: for a *write*, data travels **out** of the CPU. (This write sequence is worked out from the definitions above; check the wording against your exam board's specification.)

Exam habit: when describing a bus, say both *what it carries* and *which way it goes*. A common mistake is to say the address bus carries data — it carries only the location.

The clock: the CPU's metronome

The **clock** is a signal that ticks at a steady rate; each tick lets the CPU move one step further. Clock speed is ticks (cycles) per second: 1 MHz = 10^6 per second, 1 GHz = 10^9. The clock signal is one of the control-bus lines. [Emory University CS255 notes; TutorChase (secondary source)]

On the Uno's 16 MHz clock, one tick lasts $1/16\,000\,000\ \text{s} = 62.5\ \text{ns}$. The ATmega328P completes most simple instructions in a single tick, so it reaches *about* 1 million instructions per second (MIPS) per MHz — roughly 16 MIPS at best. Some instructions take two or more ticks. [Microchip, AVR CPU core — instruction execution timing; ATmega328P datasheet (secondary source — check against the Microchip PDF)]

Speed misconception: a higher clock speed (GHz) does not automatically mean a faster computer. Speed also depends on how much work the CPU does per clock tick, how many cores the program can use, and whether the chip slows down when hot. [Tom's Hardware; Wikipedia, instructions per cycle]

A6. The bridge to Part B: a microcontroller is a small computer on one chip

A **microcontroller** is an integrated circuit that contains a CPU, program memory, RAM and input/output pins on a *single chip*. It runs one program, starts instantly, and is designed to interact directly with sensors and

outputs.

The Arduino Uno R3 uses the **ATmega328P** microcontroller: [Arduino Uno R3 docs; Arduino Store]

Computer idea (Part A)	On the ATmega328P
CPU	8-bit processor, 16 MHz clock on the Uno
Storage (program kept without power)	32 KB flash memory (0.5 KB used by the bootloader)
RAM (working memory, lost without power)	2 KB SRAM
Extra non-volatile data	1 KB EEPROM
Input/output	14 digital pins (6 can do PWM), 6 analog inputs

Compare a laptop with gigabytes of RAM: a microcontroller is tiny but enough to read a sensor and switch a light.

Real chips differ from the model: von Neumann vs Harvard

The paper CPU in A5 keeps instructions and data in **one shared memory** on one set of buses. This design is called **von Neumann architecture**, and it is how most laptops and desktops are described in introductory courses.

The ATmega328P is different. Microchip describes the AVR as a **Harvard architecture**, "with separate memories and buses for program and data". The program sits in **flash**; variables sit in **SRAM**; each has its own bus. Because the two do not compete for the same bus, the chip can **fetch the next instruction while it executes the current one** — which is why it manages about one instruction per clock tick. [Microchip, AVR CPU core overview]

	von Neumann (paper CPU)	**Harvard (ATmega328P)**
Memory for program and data	One shared memory	Separate: flash for program, SRAM for data
Buses	One set, shared	Separate sets
Fetch and data access at the same time?	No — they take turns	Yes

(Some sources call the AVR "modified Harvard", because its program memory can also be read as data. [Wikipedia, ATmega328 (secondary source)])

Microcontroller or single-board computer?

A **single-board computer** such as a Raspberry Pi 5 is a full computer on one board: a multi-core processor at about 2.4 GHz, gigabytes of RAM, a full operating system, and a memory card or SSD for storage. [Raspberry Pi 5 product page]

Need	Usually better choice
Read sensors and switch outputs, low power, instant start	Microcontroller (e.g. Arduino Uno)
Run an operating system, a web server, a camera with image processing, or heavy computation	Single-board computer (e.g. Raspberry Pi)

Neither is "better" in general — the requirement decides. For the light indicator below, a microcontroller is the simpler fit.

Part B — Physical computing: an automatic light indicator

B1. Start with the system requirement, not the parts list

Physical component selection means choosing hardware that fits the task and the system's constraints. An engineer first defines what the system must do, then checks whether each candidate part can meet those requirements — not just picking what is popular or cheap.

Our requirement:

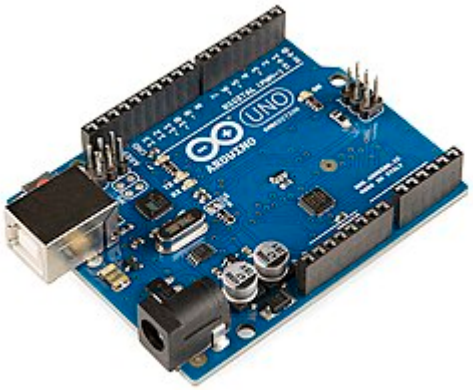
> Switch an indicator light on when a room becomes dark, and switch it off again when the room is bright.

The design needs at least:

- a **sensor** — a device that measures a physical quantity and provides a usable signal (here, brightness);
- a **controller** — here a microcontroller (A6) that reads the sensor and makes a decision;
- an **output device** — here an LED that shows the result;
- suitable resistors, wiring and a power source.

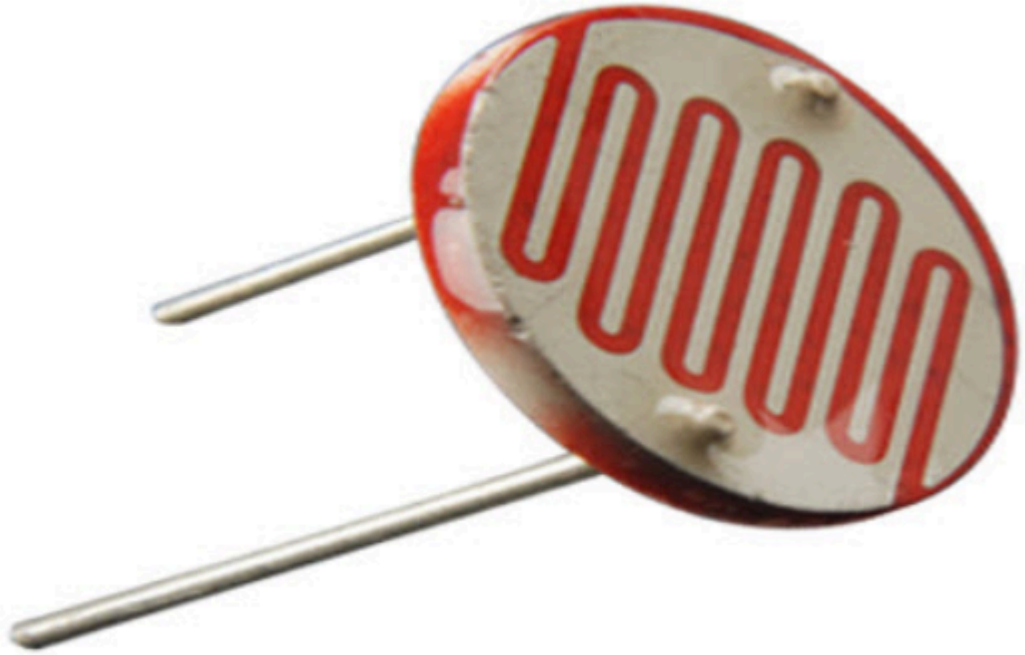
An **actuator** converts a control signal and energy into a physical effect, such as motion. In this beginner circuit the LED is treated as a simple output device; in larger systems actuators include motors, relays and solenoids (B9).

Photo: microcontroller board



An Arduino Uno R3 is a microcontroller development board. It provides digital pins, analog inputs, a USB connection and supporting circuitry. Image: Shoot stufz, Wikimedia Commons, CC BY-SA 4.0.

Photo: light sensor



A light-dependent resistor (LDR), also called a photoresistor, changes its resistance with light level. Image: Suyash Dwivedi, Wikimedia Commons, CC BY-SA 4.0.

B2. The electrical terms used in the circuit

- **Voltage (V):** electrical "push" — the potential difference between two points, measured in volts (V).
- **Current (I):** the rate of flow of electric charge, measured in amperes (A). 1 mA (milliamp) = 0.001 A.
- **Resistance (R):** opposition to current, measured in ohms (Ω). 1 kΩ = 1000 Ω.
- **Power (P):** rate of energy transfer, measured in watts (W).
- **Ground (GND):** the reference point (0 V) of the circuit, usually the supply's negative terminal.

- **Analog signal:** a signal that can take any value within a range.
- **Digital signal:** a signal that is either LOW (about 0 V) or HIGH (about the supply voltage, here 5 V).

Ohm's law links voltage, current and resistance:

$$V = IR$$

Rearranged:

$$I = \frac{V}{R} \qquad R = \frac{V}{I}$$

Electrical power:

$$P = VI = I^2R$$

B3. Series and parallel circuits

- **Series:** components in one line, one after another. The **same current** flows through each. Resistances **add**: $R_{total} = R_1 + R_2$. The supply voltage is **shared** between them.
- **Parallel:** components side by side across the same two points. Each has the **same voltage**. The total current is the sum of the branch currents, and the total resistance is **smaller** than either resistor:

$$\frac{1}{R_{total}} = \frac{1}{R_1} + \frac{1}{R_2} \qquad \text{or, for two resistors,} \qquad R_{total} = \frac{R_1 R_2}{R_1 + R_2}$$

[SparkFun, Series and Parallel Circuits]

Worked example

A 5 V supply with $R_1 = 1\ \text{k}\Omega$ and $R_2 = 2\ \text{k}\Omega$.

Series:

$$R_{total} = 1 + 2 = 3\ \text{k}\Omega \qquad I = \frac{5}{3000} \approx 1.67\ \text{mA}$$

$V_1 = 1.67\ \text{mA} \times 1\ \text{k}\Omega \approx 1.67\ \text{V}$ and $V_2 \approx 3.33\ \text{V}$. Check: $1.67 + 3.33 = 5\ \text{V}$.

Parallel:

$$R_{total} = \frac{1 \times 2}{1 + 2} \approx 0.667\ \text{k}\Omega = 667\ \Omega \qquad I = \frac{5}{667} \approx 7.5\ \text{mA}$$

Check: $5/1000 + 5/2000 = 5\ \text{mA} + 2.5\ \text{mA} = 7.5\ \text{mA}$.

The voltage divider in B6 is just a series circuit where we measure the voltage at the middle.

B4. Basic electrical safety

This unit uses only **low-voltage DC** (5 V from USB, or small batteries). **Mains electricity (220 V AC) is out of scope and must not be connected to any circuit in this module.**

Even at low voltage:

1. **Disconnect power before changing wiring.** Reconnect only after checking.
2. **Avoid short circuits** — a wire directly from 5 V to GND can overheat wires, damage the board, or trip the USB fuse.
3. **Check polarity** of LEDs, diodes, electrolytic capacitors and batteries before powering up.
4. **Hot parts mean a fault.** If anything gets hot, smells or smokes, disconnect immediately.
5. **Lithium batteries (LiPo / Li-ion)** can catch fire if punctured, short-circuited, over-charged or over-discharged. Use a proper charger and protected packs only.
6. Follow the lab's own safety rules

[UIUC ECE 110 lab safety; Adafruit, Li-ion and LiPoly batteries]

B5. Choose components using measurable criteria

Criterion	What to ask	Why it matters
Operating voltage	What supply voltage does the part need?	A wrong voltage can stop it working or damage it.
Input/output compatibility	Can the controller safely read the sensor's signal, and can its outputs drive the load?	Boards have different logic voltages and pin limits.
Current and power	How much current will the part draw or carry? Can it handle the power?	Pins, wires and supplies all have limits.
Function and performance	Is the range, response time, accuracy or speed good enough?	It must do the actual job, not just connect.
Interface	Analog voltage, digital pin, I^2C, SPI, UART…?	The controller must support it.
Environment	Temperature, moisture, vibration, light?	Conditions affect readings and reliability.
Cost and availability	Affordable? Well documented?	Undocumented parts are hard to use and maintain.

A **datasheet** is the manufacturer's technical document describing a component's pins, electrical characteristics, operating conditions and limits. Read it before choosing a part. Do not confuse **recommended operating conditions** with **absolute maximum ratings**: an absolute maximum is a damage limit to stay well away from, not a target.

Key facts for the Arduino Uno R3

From Arduino's official documentation: [Arduino Uno R3 docs; Arduino Store]

Property	Value
Operating voltage	5 V
Input voltage (barrel jack)	7–12 V recommended (6–20 V limit)
DC current per I/O pin	**20 mA** (the absolute maximum on the chip is 40 mA *(secondary source)* — never design for it)

Property	Value
Total current through the chip	about 200 mA *(secondary source)*
3.3 V pin	50 mA
USB protection	resettable fuse trips above 500 mA
Analog inputs	6 (A0–A5), 10-bit ADC
PWM pins	3, 5, 6, 9, 10, 11

These figures are for the Uno R3. They must not be assumed for every Arduino-compatible board.

Logic levels: why "5 V" vs "3.3 V" matters

A board's **logic level** is the voltage its pins use for HIGH. A classic Uno R3 uses 5 V. Many modern boards — **Raspberry Pi** and **ESP32** — use **3.3 V and are not 5 V tolerant**: connecting a 5 V signal to their inputs can damage them. Use a level shifter or a divider when mixing them. [Raspberry Pi documentation]

The newer **Arduino Uno R4** also runs at 5 V, but each pin is only rated for about **8 mA**, so the 330 Ω LED resistor used below (≈9 mA) should be raised to 470 Ω–1 kΩ on an R4. [Arduino Store, Uno R4 Minima]

Bill of Materials (BOM)

A **Bill of Materials** lists the physical components needed to build a system: what, how many, which specification and why.

Component	Qty	Specification	Purpose
Arduino Uno R3	1	5 V board with analog inputs	Reads the sensor and runs the program
Light-dependent resistor (LDR)	1	e.g. GL5528 (≈10–20 kΩ at 10 lux, ≥1 MΩ dark)	Detects brightness
Fixed resistor	1	10 kΩ, ¼ W	Forms a voltage divider with the LDR
LED	1	5 mm red, forward voltage ≈2 V, 20 mA max	Indicates the result
LED resistor	1	330 Ω, ¼ W	Limits LED current
USB cable and power source	1	5 V USB, able to supply the board	Powers the board and sensing circuit
Breadboard	1	Standard solderless	Prototyping without soldering
Jumper wires	as needed	Male–male	Connections

Selection rationale:

- **Microcontroller:** the Uno R3 has analog inputs and digital outputs, runs at 5 V, and is well documented — a good teaching platform.
- **Sensor:** an LDR is cheap and easy; its resistance changes with light.
- **Fixed resistor:** 10 kΩ is close to the LDR's resistance in normal room light, which puts the divider output near the middle of the ADC range, where changes are easiest to see.
- **LED and resistor:** visible output; 330 Ω keeps the current around 9 mA (B8), under the 20 mA pin rating.
- **Power:** USB is enough — the whole circuit draws well under 500 mA (B10).
- **Prototyping parts:** a breadboard and jumper wires make the circuit easy to change and test.

Verification before assembly:

1. Check the board's voltage and pin specifications.
2. Confirm the LDR's resistance range from its datasheet or by measuring with a multimeter.
3. Calculate the divider output at several light levels (B6).
4. Check the LED's forward voltage and current and the resistor's power rating (B8).
5. Confirm the power supply can provide enough current (B10).
6. Build, test, and only then mark the BOM as checked.

Development status: BOM drafted; specifications still need to be confirmed against the parts the team actually buys.

B6. Circuit design: an LDR in a voltage divider

An LDR's resistance **falls as light increases**. But a microcontroller cannot measure resistance directly — it measures **voltage**. To turn the resistance change into a voltage change, connect the LDR and a fixed resistor in series as a **voltage divider**.

A voltage divider is two resistors in series across a supply; the output is the voltage at the junction between them. If the output is measured across the lower resistor R_2 (the one connected to ground), the ideal equation is:

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$$

V_{in} is the supply voltage, R_1 is the top resistor and R_2 the bottom one. The formula assumes the input measuring the voltage draws almost no current — true for the Uno's analog inputs.

Wiring diagram

A 10 kΩ fixed resistor on top, the LDR on the bottom, the midpoint to analog input A0:

```
          +5 V (Uno R3)
               |
          [ R1 = 10 kΩ ]
               |
               +---------- A0 (analog input)
               |
```

```
            [ LDR = R2 ]
                  |
                GND
```

With this arrangement, **more light → lower LDR resistance → lower voltage at A0**. **Darker → higher voltage.** Beginners often expect the opposite, so point this out. Swapping the two parts would reverse the direction.

Important: a voltage divider is for *sensing* only. Do not use one to power a load; use a proper voltage regulator instead.

Worked example: estimate the sensor voltage

Assume a 5 V supply, $R_1 = 10\ \text{k}\Omega$, and two illustrative LDR readings:

- **Dim room:** $R_2 = 100\ \text{k}\Omega$
- **Bright room:** $R_2 = 2\ \text{k}\Omega$ (roughly a well-lit room, a few hundred lux, for a GL5528-type LDR)

These are teaching values. Real LDRs vary between models, so measure your own part or check its datasheet. In **true darkness** a GL5528 can exceed 1 MΩ. [GL55 series datasheet]

Case A — dim:

$$V_{out} = 5 \times \frac{100}{10 + 100} = 5 \times \frac{100}{110} \approx 4.55\ \text{V}$$

Case B — bright:

$$V_{out} = 5 \times \frac{2}{10 + 2} = 5 \times \frac{2}{12} \approx 0.83\ \text{V}$$

Interpretation: the dim example gives a much higher voltage than the bright one, so the controller can tell them apart.

B7. From voltage to a number: the ADC

An **ADC (analog-to-digital converter)** turns an input voltage into a whole number that a program can use. The Uno R3's ADC is **10-bit**: it gives $2^{10} = 1024$ possible codes, numbered **0 to 1023**. With the default 5 V reference, `analogRead()` returns:

$$\text{ADC code} = \left\lfloor \frac{V_{in}}{5\ \text{V}} \times 1024 \right\rfloor \quad (\text{maximum } 1023)$$

Each step is $5\ \text{V} / 1024 \approx 4.9\ \text{mV}$. A reading of 1023 means "5 V minus one step or more". Many tutorials use $\times 1023$ instead; the difference is under 0.1 % — smaller than the variation in a USB 5 V supply — so either is fine for estimates. [Arduino analogRead reference; ATmega328P datasheet]

Using the worked example:

- Dim: $4.55 / 5 \times 1024 \approx 931$
- Bright: $0.83 / 5 \times 1024 \approx 170$
- True darkness (≈1 MΩ LDR): $4.95 / 5 \times 1024 \approx 1013$

These are ideal estimates. Real readings differ because of the actual LDR, the real supply voltage, resistor tolerance and electrical noise.

Resolution, accuracy and precision

- **Resolution:** the smallest change the ADC can show — here ≈4.9 mV, one step.
- **Accuracy:** how close the reading is to the true value. It depends on the reference voltage, component tolerances and ADC errors — *not* just on resolution.
- **Precision (repeatability):** how much repeated readings of the same thing agree.

A 10-bit ADC has fine resolution, but if the 5 V USB supply is really 4.8 V, every reading is about 4 % off. **High resolution does not guarantee high accuracy.**

Sampling

Each `analogRead()` takes about **100 µs**, so the Uno can take roughly 10,000 readings per second — far faster than room light changes. A delay of 50–100 ms between readings is plenty for this project. [Arduino analogRead reference]

Smoothing, calibration and hysteresis

- **Smoothing:** average several readings to reduce noise. Arduino's *Smoothing* example averages the last 10 readings. [Arduino Smoothing example]
- **Calibration:** measure the real minimum and maximum in *your* room, then set the threshold between them. Arduino's *Calibration* example records min and max during the first 5 seconds, then uses `map()` to rescale. [Arduino Calibration example]
- **Hysteresis:** if the light level sits right at the threshold, noise makes the LED flicker on and off. Use **two thresholds** — for example turn ON above 550 but only turn OFF below 450. The gap between them stops the flicker. This is a standard control technique; see the second code example in B11.

B8. Add an output: an LED indicator

An **LED (light-emitting diode)** converts electrical energy into light. It only conducts one way: current flows from the **anode** (+, longer leg) to the **cathode** (−, shorter leg, flat edge on the case).

An LED **must** have a series resistor to limit its current. A typical red LED has a forward voltage of about 2 V (range 1.8–2.2 V) and a maximum of 20 mA. [SparkFun, basic red LED]

For a 5 V output, a 2 V LED and a target of 10 mA, the resistor must drop:

$$V_R = 5 - 2 = 3 \text{ V}$$

$$R = \frac{V_R}{I} = \frac{3}{0.010} = 300 \ \Omega$$

The nearest common value above that is **330 Ω**:

$$I = \frac{3}{330} \approx 0.0091 \text{ A} = 9.1 \text{ mA}$$

That is below the Uno R3's 20 mA pin rating. **Resistor power check:**

$$P = I^2 R = 0.0091^2 \times 330 \approx 0.027\ \text{W} = 27\ \text{mW}$$

That is far under a standard ¼ W (250 mW) resistor's rating.

Wiring: **digital pin 9 → 330 Ω resistor → LED anode → LED cathode → GND.**

B9. Actuators: when a pin is not enough

A microcontroller pin is a **signal**, not a power supply. Rated at 20 mA, it can light an LED but **cannot** run most motors, relays or large buzzers.

Load	Typical current	Drive directly from a pin?
Small LED + resistor	5–15 mA	Yes
Active buzzer (small)	~10–30 mA	Sometimes — check its datasheet
Relay coil (5 V)	~70–100 mA	**No** — needs a transistor driver
Small DC motor	100 mA – several A	**No** — needs a transistor/MOSFET or motor driver
Hobby servo	up to ~1 A while moving	Signal yes, **power no** — use a separate supply

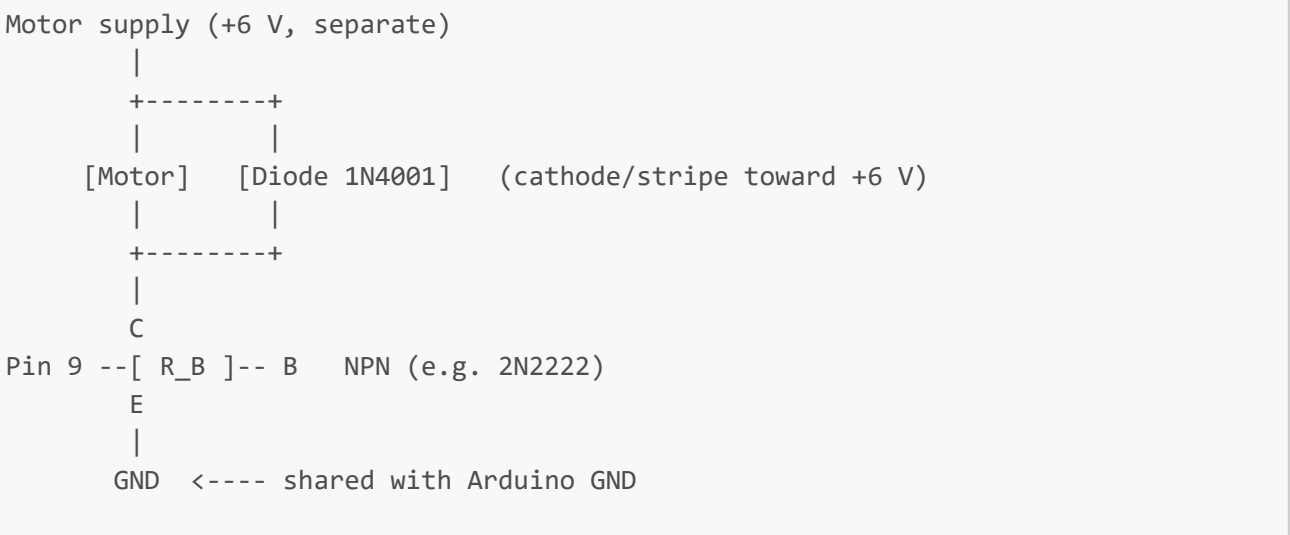
Active vs passive buzzer: an *active* buzzer has its own oscillator and beeps when given DC power; a *passive* buzzer needs the microcontroller to generate a tone (for example with `tone()`).

Why motors and relays are special: inductive loads and flyback diodes

Motors and relay coils are **inductive loads** — they store energy in a magnetic field. When switched off suddenly, they produce a **voltage spike** that can destroy the transistor driving them. A **flyback diode** (for example a 1N4001) placed **across the load, pointing toward the positive supply** (cathode to +) gives that energy a safe path. [Wikipedia, flyback diode]

A transistor as a switch

A **transistor** lets a small current (from the pin) control a much bigger current (through the load). In a **low-side switch** the transistor sits between the load and ground:



Common ground: the motor supply's negative and the Arduino's GND **must be connected**, or the transistor has no reference and will not switch.

Worked example: choose the base resistor

A small motor draws 150 mA. For an NPN transistor to be **fully on** (saturated), a common rule of thumb is to give the base about **one tenth** of the load current ("forced beta" of 10):

$$I_B = \frac{150\ \text{mA}}{10} = 15\ \text{mA}$$

The base–emitter junction drops about 0.7 V, so:

$$R_B = \frac{5 - 0.7}{0.015} \approx 287\ \Omega$$

Use **270 Ω or 330 Ω**. With 330 Ω, $I_B = 4.3/330 \approx 13\ \text{mA}$ — enough to switch the motor, and safely under the 20 mA pin rating.

A common mistake is using 1 kΩ: that gives only ≈4.3 mA, which may not fully turn the transistor on at 150 mA. It is fine for a 70 mA relay coil, though. [SparkFun, Transistors; Adafruit, DC motors lesson]

For loads above about **200 mA**, the base current would exceed the pin limit. Use a **logic-level N-channel MOSFET** instead (for example IRLB8721, which fully switches with 5 V on its gate), with a small gate resistor (≈100–220 Ω) and a 10 kΩ pull-down from gate to GND so it stays off at power-up. [Adafruit, IRLB8721]

Motor-driver modules

To run a motor **forwards and backwards**, use an **H-bridge** driver module rather than building one:

Driver	Motor voltage	Current per channel	Notes
L298N	up to 46 V	2 A (4 A total)	Old design; wastes ~1.4–3 V as heat
TB6612FNG	4.5–13.5 V	1.2 A (3 A peak)	Efficient; protection diodes built in
DRV8833	2.7–10.8 V	1.5 A (2 A peak)	Efficient; good for small robots

[ST L298; Adafruit TB6612; TI DRV8833]

PWM: controlling brightness and speed

Digital pins are only ON or OFF. **PWM (pulse-width modulation)** switches a pin on and off very quickly. The **duty cycle** is the percentage of time it is ON:

$$\text{duty cycle} = \frac{t_{on}}{t_{on} + t_{off}} \times 100\%$$

On average, a 5 V pin at 50 % duty cycle behaves like about 2.5 V, so an LED looks dimmer or a motor turns slower.

On the Uno R3, `analogWrite(pin, value)` takes a value from **0 to 255**: 0 = always off, 128 ≈ 50 %, 255 = always on. It works only on pins **3, 5, 6, 9, 10, 11**, at about 490 Hz (980 Hz on pins 5 and 6). [Arduino analogWrite reference; SparkFun, PWM]

Servos

A **hobby servo** turns to a commanded angle. It reads a pulse every 20 ms (50 Hz): about 1 ms ≈ one end, 1.5 ms ≈ centre, 2 ms ≈ the other end. The Arduino `Servo` library generates these pulses. Power anything bigger than a micro-servo from a **separate 5–6 V supply** (allow about 1 A per servo) with a **shared ground**. On the Uno, using the `Servo` library disables `analogWrite()` on pins 9 and 10, so move a PWM LED to another PWM pin (for example 5 or 6) if you add a servo. [SparkFun, Hobby Servo Tutorial; Arduino Servo library]

B10. Power budget

A **power budget** adds up the current every part needs and checks it against what the supply can give.

Item	Current (approx.)
Arduino Uno R3 board itself	~50 mA *(estimate — measure to confirm)*
LED via 330 Ω	9 mA
LDR divider (worst case, bright: $5\ \mathrm{V}/12\ \mathrm{k\Omega}$)	0.4 mA
Total	**≈60 mA**

USB can supply up to 500 mA, so the light indicator has a large safety margin. A motor or servo would change that: **power motors from their own supply**, never from the Uno's 5 V pin.

B11. Connect the sensor reading to the output

This Arduino sketch reads the divider and turns the LED on when it is dark. The threshold `500` is only a starting point; **calibrate it** using the Serial Monitor readings in your own room.

```
const int sensorPin = A0;
const int ledPin = 9;
const int darkThreshold = 500;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int lightValue = analogRead(sensorPin);
  Serial.println(lightValue);

  // With the LDR as the lower divider resistor,
  // darker conditions produce a higher reading.
  if (lightValue > darkThreshold) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
```

```
  delay(100);
}
```

Improved version: smoothing and hysteresis

```
const int sensorPin = A0;
const int ledPin = 9;
const int onThreshold = 550;   // turn LED on when reading rises above this
const int offThreshold = 450;  // turn LED off only when reading falls below this
const int samples = 10;

bool ledOn = false;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Smoothing: average several readings to reduce noise.
  long total = 0;
  for (int i = 0; i < samples; i++) {
    total += analogRead(sensorPin);
    delay(5);
  }
  int lightValue = total / samples;
  Serial.println(lightValue);

  // Hysteresis: two thresholds stop flicker near the boundary.
  if (!ledOn && lightValue > onThreshold) {
    ledOn = true;
  } else if (ledOn && lightValue < offThreshold) {
    ledOn = false;
  }
  digitalWrite(ledPin, ledOn ? HIGH : LOW);

  delay(50);
}
```

Both sketches are written for the circuit in B6 and B8 on an Uno R3. They have **not yet been tested** by the team on real hardware. Check the pinout of your exact board before building.

Ready-to-open copies of both sketches, plus a calibration sketch and two extensions (PWM dimming, push-button mode switch), are in arduino/. Uploading steps are in beginner-lab-skills.md, L4.

B12. Overall system diagram

```
Room light
    |
```

```
    v
LDR + 10 kΩ resistor        (physical input -> voltage)
    |
    v
Analog input A0 / ADC        (voltage -> number 0–1023)
    |
    v
Microcontroller program      (fetch–decode–execute: compare with threshold)
    |
    v
Digital output pin 9         (number -> HIGH/LOW voltage)
    |
    v
330 Ω resistor + LED         (electrical output -> light)
```

The parts form a **chain**: wrong wiring changes the measurement, a wrong threshold causes wrong decisions, and a missing LED resistor can damage the LED or the pin. Notice how Part A appears in the middle — the microcontroller's CPU fetches, decodes and executes the `if` statement thousands of times a second.

B13. Learner activity

Try the questions before reading the answers. Show your calculations for Questions B and E.

Question A — Component selection. You need to build the automatic light indicator. Name four pieces of information you would check before choosing the board, LDR, fixed resistor and LED, and say why each matters.

Question B — Voltage divider. The input is 5 V, the fixed resistor is 10 kΩ and the LDR is 10 kΩ. Calculate the output voltage at the midpoint.

Question C — Interpret a measurement. On the Uno R3 circuit in B6, one condition reads about 900 and another about 180. Which is more consistent with darkness, and why?

Question D — Actuator safety. A teammate proposes connecting a small DC motor directly to a GPIO pin. Explain why this is unsuitable and name the kind of interface to use instead.

Question E — Series and parallel. Two 2 kΩ resistors are connected to 5 V. Find the total resistance and total current (i) in series and (ii) in parallel.

Question F — PWM. What duty cycle does `analogWrite(9, 64)` give, and roughly what average voltage?

Question G — Paper CPU. Using the paper CPU in A5, change address 5 to `9`. What value ends up in address 6, and how many fetch–decode–execute cycles run?

B14. Answers and expected outcomes

Answer A. Possible answers: operating and logic voltage; current and power limits; sensor resistance range; ADC/interface compatibility; LED forward voltage and current; environment; cost and availability; the datasheets. A good answer explains *why* each matters, not just lists terms.

Answer B.

$$V_{out} = 5 \times \frac{10}{10 + 10} = 2.5 \text{ V}$$

Equal resistances split the supply in half.

Answer C. About 900. In this wiring, darkness raises the LDR resistance, which raises the midpoint voltage and therefore the ADC code. Real thresholds depend on your components and room — calibrate rather than treating 900 as universal.

Answer D. A GPIO pin is rated for about 20 mA and is meant for signals; a motor may need hundreds of milliamps, and as an inductive load it creates voltage spikes when switched. Use a transistor or logic-level MOSFET switch with a flyback diode, or a motor-driver module, powered from a separate supply with a shared ground.

Answer E. (i) Series: $R = 4\ \text{k}\Omega$, $I = 5/4000 = 1.25\ \text{mA}$. (ii) Parallel: $R = (2 \times 2)/(2 + 2) = 1\ \text{k}\Omega$, $I = 5/1000 = 5\ \text{mA}$.

Answer F. $64/255 \approx 25\%$ duty cycle, so about $0.25 \times 5 \approx 1.25$ V on average.

Answer G. $7 + 9 = 16$ is stored in address 6. Four cycles run (`LOAD`, `ADD`, `STORE`, `HALT`) — changing data does not change the number of instructions.

B15. Troubleshooting checklist

Change one thing at a time:

1. Check the board pinout and supply voltage against the official documentation.
2. Confirm the fixed resistor and LDR are in series and A0 connects to their junction.
3. Check all grounds are connected together.
4. Print readings to the Serial Monitor and test bright and dark conditions.
5. Adjust the threshold only after seeing real readings.
6. Check the LED's orientation (long leg to the resistor/pin side) and that it has a series resistor.
7. If the LED flickers near the threshold, add smoothing and hysteresis (B11).
8. If adapting to another board, check its logic voltage and pin current limits first (B5).

Part C — Going further: talking to other devices, capacitors, datasheets and sustainability

Part B built one complete circuit. Part C extends it in four directions: how a microcontroller talks to *digital* sensors and other computers (C1–C2), the one common component Part B did not use (C3), how to read a real datasheet line by line (C4), and what happens to the parts afterwards (C5).

C1. Serial communication: UART, I²C and SPI

B5 lists "I²C, SPI, UART" as interface types. All three are **serial** interfaces: they send data **one bit at a time** over a few wires, instead of using one wire per bit. They differ in how many wires they need, how many devices can share them, and how fast they go.

UART — two devices, no clock wire

UART (universal asynchronous receiver-transmitter) links **exactly two devices**. Each has a **TX** (transmit) and an **RX** (receive) pin, and the wires **cross**: TX of one goes to RX of the other. There is no clock wire, so both ends must agree on the speed in advance — the **baud rate**, in bits per second (9600 and 115200 are common). [SparkFun, Serial Communication]

Each byte is sent as a **frame**: a start bit, 8 data bits, and a stop bit. This common setting is called **8N1** (8 data bits, no parity bit, 1 stop bit), and it is Arduino's default for `Serial.begin()`. Because of the start and stop bits, **every byte costs 10 bits on the wire**, so 9600 baud carries about 960 bytes per second. [SparkFun, Serial Communication; Arduino, Serial.begin()]

On the Uno R3 the hardware UART is on **pins 0 (RX) and 1 (TX)**, and the same pins also go to the board's USB-to-serial chip (an ATmega16U2 *(secondary source)*). That is why the Serial Monitor and anything wired to pins 0/1 interfere with each other. **Never** connect these pins to an old RS-232 serial port, which uses about ±12 V. On the Uno R4, pins 0/1 are a separate port (`Serial1`), so the clash goes away. [SparkFun, Serial Communication; Arduino, Serial reference]

I²C — many addressed devices on two shared wires

I²C (inter-integrated circuit, said "I-squared-C") uses two wires shared by every device on the bus: **SDA** (data) and **SCL** (clock). Each device has its own **7-bit address**, written in hex, so the controller can talk to one at a time. On the Uno R3, SDA is **A4** and SCL is **A5** (also on the SDA/SCL header pins). Arduino's library is `Wire`. [SparkFun, I2C; Arduino, Wire reference]

- **Pull-up resistors are required.** Devices can only pull the lines *low*; a resistor from each line to the supply pulls them *high* again. SparkFun suggests starting at 4.7 kΩ. Many breakout boards already include them — check before adding more. [SparkFun, I2C]
- **Speed:** 100 kHz (standard) or 400 kHz (fast), set with `Wire.setClock()`. [Arduino, Wire.setClock()]
- **Distance:** short runs only, about 2–3 m. [SparkFun, I2C]
- **How many devices?** 112 addresses are usable in theory; in practice, around seven on one bus is a sensible limit. [SparkFun, I2C]
- **Example addresses:** a **BH1750 light sensor** is `0x23` (or `0x5C` if its ADDR pin is high) *(secondary source — from driver code, check the ROHM datasheet)*; an Adafruit **BME280** temperature/pressure/humidity breakout is `0x77` (or `0x76`). [Adafruit, BME280 guide] Because the addresses differ, both can share the same two wires.

Link to the project: a BH1750 is a *digital* light sensor that reports brightness directly as a number over I²C, instead of needing a voltage divider and the ADC. Comparing it with the LDR is a good component-selection discussion (B5): more accurate and calibrated, but more expensive and more complex to use.

SPI — more wires, more speed

SPI (serial peripheral interface) uses three shared wires plus **one chip-select (CS) wire for each device**:

Signal	Job	Uno R3 pin
SCK	Clock from the controller	13
COPI (older name MOSI)	Controller out, peripheral in	11

Signal	Job	Uno R3 pin
CIPO (older name MISO)	Controller in, peripheral out	12
CS (older name SS)	Selects one device: **LOW = selected**	10 (or any spare pin, one per device)

The same signals also appear on the 6-pin ICSP header. Data can flow both ways at once (full duplex). On the Uno the SPI clock can run from 125 kHz up to 8 MHz — much faster than I²C — but only over short distances (a few feet). Arduino's library is `SPI`. *Naming note:* Arduino uses COPI/CIPO, SparkFun uses PICO/POCI, and many older modules still print MOSI/MISO — they are the same lines. [SparkFun, SPI; Arduino, SPI reference]

The Adafruit BME280 breakout can be wired as **either** I²C or SPI — the bus is a choice made by the designer, not a property of the measurement. [Adafruit, BME280 guide]

Comparing the three

	UART	I²C	SPI
Signal wires (plus GND)	2 (TX/RX, crossed)	2 (SDA/SCL, shared)	3 shared + 1 CS per device
Clock wire?	No — agree a baud rate	Yes (SCL)	Yes (SCK)
How a device is chosen	Only two devices	By address	By its CS wire
Typical speed on Uno	9600–115 200 baud	100 / 400 kHz	125 kHz – 8 MHz
Uno R3 pins	0 (RX), 1 (TX)	A4 (SDA), A5 (SCL)	10–13 + ICSP
Arduino library	`Serial`	`Wire`	`SPI`
Typical use	PC link, GPS, Bluetooth modules	Several slow sensors, small displays	Fast devices: SD cards, displays

Logic levels: 3.3 V modules on a 5 V Uno

B5 warns about 3.3 V *boards*. The more common beginner case is a 3.3 V **sensor module** on a 5 V Uno. For example, the bare BME280 chip runs at 1.71–3.6 V and is **not** 5 V-tolerant *(secondary source)*, while Adafruit's breakout adds a regulator and level shifting so it is safe on 5 V. [Adafruit, BME280 guide] Before connecting any module, check:

1. Does its power pin (VIN) have a **regulator**, or must it get 3.3 V?
2. Are its data pins **level-shifted**, or 3.3 V only?
3. For I²C, which voltage do the **pull-up resistors** connect to?

If the answer is unclear, use a **logic-level converter** between the Uno and the module. [SparkFun, I2C]

C2. Sending readings to another device: CSV over serial

The sketches already print one value per line. Printing several values separated by commas makes a **CSV** (comma-separated values) line:

```
Serial.print(lightValue);
Serial.print(',');
Serial.println(ledOn ? 1 : 0);   // e.g. "734,1" then a new line
```

The same stream can then be read in three ways:

1. **Arduino Serial Plotter** — draws one line on the graph for each comma-separated column; values can also be labelled as `name:value` (sketch 02 does this). *(Format rule from secondary sources.)*
2. **A Python program** on the computer, using the `pyserial` library: `serial.Serial('COM3', 9600, timeout=1)` then `readline()` returns one line up to the newline. Set a **timeout**, or `readline()` can wait forever. [pyserial documentation] `println()` ends each line with `\r\n`, so strip it off (`line.strip()`) before splitting on commas.
3. **A spreadsheet**, once the lines are saved to a `.csv` file.

Bridge to the Networks unit: this tiny format already has the ideas a network protocol needs — a **newline marks where each record ends** (framing), a **comma separates the fields**, and an optional first line can name the fields. Packets, addresses and data formats in the Networks unit build on exactly these ideas.

C3. Capacitors

A **capacitor** stores electric charge, a little like a tiny rechargeable reservoir that fills and empties very quickly. Capacitance is measured in **farads (F)**; real parts are usually in **pF** (picofarads, 10^{-12} F), **nF** (10^{-9} F) or **µF** (10^{-6} F). Every capacitor also has a **voltage rating**; exceeding it will usually destroy the part. [SparkFun, Capacitors]

Type	Typical values	Polarity	Typical use
Ceramic	pF to a few µF	None — either way round	Decoupling, filtering
Aluminium electrolytic	1 µF – 1 mF and more	**Polarised:** stripe marks −, longer leg is +	Bulk storage near motors/servos

An electrolytic capacitor connected the wrong way round can "fail spectacularly (making a pop and bursting open)". Always check the stripe before powering up (B4). [SparkFun, Capacitors]

Reading the 3-digit code on small ceramic capacitors: the first two digits are the value and the third is how many zeros to add, in **picofarads**. So **104** = 10 followed by 4 zeros = 100 000 pF = **100 nF = 0.1 µF** (not 104 pF). A letter after it gives the tolerance: J = ±5 %, K = ±10 %, M = ±20 %. *(secondary source)* This works like the resistor colour code in beginner-lab-skills.md L3.

Decoupling and bulk capacitors

- **Decoupling capacitor:** a **0.1 µF ceramic** placed **as close as possible** to a chip's power pin. When the chip suddenly draws current, this small local store supplies it, keeping the supply voltage steady. Manufacturers (Microchip, Texas Instruments) consistently recommend this. [Microchip, decoupling capacitors; TI THS4281 datasheet]
- **Bulk capacitor:** a larger electrolytic (tens to hundreds of µF) that smooths bigger, slower dips. This matters as soon as a **servo** is added (B9): a servo starting up can pull the supply down far enough to

reset the Arduino. Adafruit's fix is a capacitor of **470 µF or more** between 5 V and GND, longer (+) leg to 5 V. [Adafruit, Arduino Lesson 14 — servo motors]

A capacitor on the sensor input (optional)

A 100 nF ceramic from A0 to GND forms an **RC filter** with the divider. Its **time constant** is

$$\tau = RC = 10\ \text{k}\Omega \times 100\ \text{nF} = 10\,000 \times 0.000\,000\,1 = 0.001\ \text{s} = 1\ \text{ms}$$

It is harmless and can reduce fast electrical noise. It is **not needed**: Microchip says the AVR ADC works best with a source resistance of about 10 kΩ or less, and this divider is always below that. [Microchip, AVR analog input circuitry (secondary — generic AVR page)] It also **cannot** remove the 100/120 Hz flicker of mains lighting, because it only filters changes faster than about 160 Hz — so the software averaging in B7/B11 is still the main smoothing method. A much bigger capacitor (100 µF, τ ≈ 1 s) would make the readings sluggish.

C4. Reading a real datasheet: the GL5528 LDR and a red LED

B5 says "read the datasheet". Here is what that looks like for the two parts in this project. Simple parts' datasheets usually contain: a description, dimensions, **absolute maximum ratings**, **electrical/optical characteristics** (with *min*, *typ* and *max* columns and the *test conditions*), and **graphs**.

Datasheet line	GL5528 LDR	Kingbright WP7113ID red LED	In our circuit	Safe?
Voltage limit	150 V max	5 V **reverse** max	≤ 5 V across the LDR	Yes, far below
Power limit	100 mW	75 mW	LDR ≤ 0.63 mW; LED ≈ 18 mW	Yes
Current limit	—	30 mA continuous; 160 mA brief pulses only	≈ 8–9 mA	Yes, about ⅓ of max
Key characteristic	10–20 kΩ at 10 lux; ≥ 1 MΩ dark; γ = 0.6	Forward voltage 1.9 V typ, 2.3 V max (at 10 mA); 30° viewing angle	—	—
Timing / temperature	20 ms rise, 30 ms decay; −30 to +70 °C	−40 to +85 °C, with a derating curve	Room temperature	Yes

[GL5528 datasheet (Rapid Electronics / TruOpto); Kingbright WP7113ID datasheet] *The last two columns are our own calculations from those values.*

Four lessons from this table:

1. **Design to min/max, not "typ".** "Typ" is a typical part; yours may differ. With 330 Ω, the LED current is $(5 - 1.9)/330 \approx 9.4\ \text{mA}$ at typical forward voltage, but $(5 - 2.3)/330 \approx 8.2\ \text{mA}$ at the maximum. Both are safe. The LDR's 10–20 kΩ spread at the same light level — a factor of 2 between parts — is the datasheet's own reason why B7 tells you to **calibrate every build**.
2. **Read the graphs, not just the headline numbers.** The LED's 30 mA limit applies at about 25 °C; its derating curve falls to about 5 mA near 85 °C. A part in a hot enclosure has a lower limit.

3. **Gamma (γ) explains how much the reading changes.** γ = 0.6 means ten times more light divides the resistance by about $10^{0.6} \approx 4$. So at 100 lux the LDR is roughly 2.5–5 kΩ — consistent with the "bright room" 2 kΩ value used in B6.
4. **Check the specific part you bought.** The BOM in B5 lists "20 mA max", which is from SparkFun's generic red LED; the WP7113ID is rated 30 mA. Neither is wrong — they are different parts. The BOM should name the exact part number bought and quote **its** datasheet.

Two smaller points: an LED connected backwards on a 5 V pin sits right at its 5 V reverse-voltage limit, so fix it rather than leaving it; and Kingbright warns not to bend the LED legs "more than twice". *(The GL5528 sheet used here is a distributor's copy; one other site gives γ = 0.7 — use the datasheet's 0.6.)*

C5. Sustainability: e-waste and responsible parts

The scale of the problem. The *Global E-waste Monitor 2024* reports **62 million tonnes** of electronic waste generated worldwide in 2022 — up 82 % since 2010 and heading for 82 million tonnes by 2030. Only **22.3 %** was documented as properly collected and recycled, and e-waste is growing **five times faster** than documented recycling. The metals in it were worth about US$91 billion. It defines e-waste as "any discarded product with a plug or battery". [UNITAR, Global E-waste Monitor 2024 press release]

RoHS links sustainability back to the datasheet. The EU **RoHS** directive (2011/65/EU, extended by 2015/863) restricts ten hazardous substances in electronics, including **lead**, **cadmium** and **mercury**. [European Commission, RoHS directive] Look at this project's parts:

- The Kingbright LED is marked **RoHS compliant** and recommends **lead-free** (SAC 305) solder. [Kingbright WP7113ID datasheet]
- The GL5528's sensing layer is **CdS — cadmium sulphide**. [GL5528 datasheet] The LDR therefore contains cadmium and must go to **e-waste collection, never the normal bin**. Reuse it between projects instead of throwing it away.

Lab habits:

- **Reuse first.** C4 shows these parts run far below their ratings, so they can be reused many times. Keep a labelled parts-return tray, and prefer breadboard builds over soldering where possible.
- **Batteries never go in household rubbish or recycling bins.** Tape the terminals or bag each battery, and take them to a battery collection point. [US EPA, used household batteries] Follow your local authority's rules.
- **Dead boards, cables and modules** go to an e-waste collection point.
- **Buy from authorised distributors.** Counterfeit parts are often old or refurbished parts sold as new; they may not match their datasheet at all, which defeats C4. *(secondary source)*

C6. Practice questions

Question M — Hexadecimal. Convert the I²C address `0x5C` to binary and to denary.

Question N — Buses. During the execute step of `STORE 6` in the paper CPU, what is on the address bus, what is on the data bus, and which control signal is sent?

Question O — Choosing a bus. You want to add both a BH1750 light sensor and a BME280 sensor to the Uno, using as few pins as possible. Which interface would you choose, how many signal wires are needed, and how does the Uno tell the two sensors apart?

Question P — Capacitors. A ceramic capacitor is marked `104`. What is its value in nF? What time constant does it give with a 10 kΩ resistor?

Question Q — Datasheet. Using the C4 table, a learner wants to replace the 330 Ω LED resistor with 100 Ω "to make it brighter". Using the maximum forward voltage (2.3 V) and the *typical* one (1.9 V), is this within the WP7113ID's absolute maximum?

Question R — Harvard. Give one difference between the paper CPU in A5 and the ATmega328P, and explain why it helps the ATmega328P run faster.

Answers

Answer M. 5 = 0101, C = 1100, so `0x5C` = **0101 1100** = $5 \times 16 + 12 = 92$.

Answer N. Address bus: **6** (from the MAR). Data bus: **12** (from the ACC via the MDR, going *out* to memory). Control signal: **write**.

Answer O. I²C: only **2 signal wires** (SDA on A4, SCL on A5) plus power and GND, shared by both sensors. The Uno tells them apart by their **addresses** (BH1750 `0x23`, BME280 `0x77`).

Answer P. 10 + four zeros = 100 000 pF = **100 nF**. $\tau = 10\,000 \times 100 \times 10^{-9} = 0.001\ \text{s} = \mathbf{1\ ms}$.

Answer Q. $(5 - 2.3)/100 = 27\ \text{mA}$; $(5 - 1.9)/100 = 31\ \text{mA}$. With a typical LED the current would be **above the 30 mA absolute maximum** — and above the Uno R3's 20 mA pin rating in either case. **No**: keep 330 Ω. (Absolute maximum is a damage limit, not a target — B5.)

Answer R. The paper CPU keeps program and data in **one shared memory** (von Neumann); the ATmega328P has **separate memories and buses** for program (flash) and data (SRAM) (Harvard). It can fetch the next instruction while executing the current one, so it completes about one instruction per clock tick.

How this content becomes the three resources

Content	Study Guide + Test Bank	PPT slide visual	Video segment
A1–A2 components	Component table + matching questions	Labelled computer diagram	"Follow a photo from storage to screen"
A5 fetch–decode–execute	Paper-CPU trace + Question G	Animated cycle diagram (PC → IR → CU → ALU)	Learners acting as CPU and memory
B6–B7 divider and ADC	Worked example + Questions B, C	Wiring diagram with voltage labels	Live build reading values on the Serial Monitor
B9 actuators	Base-resistor example + Question D	"Pin vs motor" current comparison chart	Why a motor needs a driver (safe demo)

Use the same terms, values and diagrams in all three, so a correction made here is copied into every resource.

Further development plan

- [ ] Peer-review Part A against the course lecture definitions (team member to be assigned).
- [ ] Peer-check every calculation in B3–B10 and record the outcome in the project log.
- [ ] Build and test the B6/B8 circuit and both sketches in the lab; record real readings and update thresholds.
- [ ] Confirm the 40 mA / 200 mA chip limits and ADC accuracy against the Microchip ATmega328P datasheet.
- [ ] Measure the Uno's own current draw to replace the ~50 mA estimate.
- [ ] Draw original diagrams for the slides (component diagram, fetch–decode–execute cycle, wiring diagram).
- [ ] Coordinate with the Networks and Behaviorist AI owners on the shared smart-room example.
- [ ] Update the status table honestly after each review.
- [ ] Check every *(secondary source)* item in the new A3/A5/A6 subsections and Part C against the primary documents listed in reports/Hardware unit gap topics.md (mainly the Microchip ATmega328P datasheet, the Cambridge 0478 syllabus, and the ROHM BH1750 and Bosch BME280 datasheets).
- [ ] Update the BOM (B5) with the exact LED part number bought and its datasheet current rating (see C4).

References

Computer hardware (Part A)

1. Arduino. "UNO R3" — specifications and documentation. https://docs.arduino.cc/hardware/uno-rev3/
2. Arduino Store. "Arduino Uno Rev3" — technical specifications. https://store.arduino.cc/products/arduino-uno-rev3
3. Microchip. *ATmega328P Datasheet*. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
4. Raspberry Pi Ltd. "Raspberry Pi 5" — product page. https://www.raspberrypi.com/products/raspberry-pi-5/
5. Nisan, N. and Schocken, S. *Nand to Tetris*. https://www.nand2tetris.org/
6. Save My Exams. "The Fetch-Decode-Execute Cycle" (IGCSE Computer Science revision notes). https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle
7. Computer Science Wiki. "The machine instruction cycle." https://computersciencewiki.org/index.php/The_machine_instruction_cycle
8. Save My Exams. "Truth Tables" (GCSE Computer Science). https://savemyexams.com/gcse/computer-science/aqa/20/revision-notes/5-boolean-logic/boolean-logic/truth-tables
9. NIST. "Prefixes for binary multiples." https://physics.nist.gov/cuu/Units/binary.html
10. Corsair. "Volatile vs non-volatile memory: what's the difference?" (vendor explainer). https://www.corsair.com/uk/fi/explorer/diy-builder/memory/volatile-vs-non-volatile-memory-whats-the-difference/
11. Tom's Hardware. "What is clock speed?" https://www.tomshardware.com/news/clock-speed-definition,37657.html

Circuits, sensors and actuators (Part B)

12. Arduino. `analogRead()` reference. https://docs.arduino.cc/language-reference/en/functions/analog-io/analogRead/
13. Arduino. `analogWrite()` reference. https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWrite/
14. Arduino. "Analog Input" example. https://docs.arduino.cc/built-in-examples/analog/AnalogInput/
15. Arduino. "Smoothing" example. https://docs.arduino.cc/built-in-examples/analog/Smoothing/
16. Arduino. "Calibration" example. https://docs.arduino.cc/built-in-examples/analog/Calibration/
17. Arduino. Servo library. https://docs.arduino.cc/libraries/servo/
18. Arduino Store. "UNO R4 Minima." https://store-usa.arduino.cc/products/uno-r4-minima
19. Raspberry Pi Ltd. Raspberry Pi hardware documentation (GPIO voltage). https://www.raspberrypi.com/documentation/computers/raspberry-pi.html
20. GL55 Series Photoresistor datasheet. https://cdn.shopify.com/s/files/1/0699/1344/4452/files/GL55_Series_Photoresistor.pdf
21. SparkFun Learn. "Voltage Dividers." https://learn.sparkfun.com/tutorials/voltage-dividers/introduction
22. SparkFun Learn. "Series and Parallel Circuits." https://learn.sparkfun.com/tutorials/series-and-parallel-circuits/all
23. SparkFun Learn. "Transistors." https://learn.sparkfun.com/tutorials/transistors/all
24. SparkFun Learn. "Pulse Width Modulation." https://learn.sparkfun.com/tutorials/pulse-width-modulation/all
25. SparkFun Learn. "Hobby Servo Tutorial." https://learn.sparkfun.com/tutorials/hobby-servo-tutorial/all
26. SparkFun. "LED – Basic Red 5mm." https://www.sparkfun.com/led-basic-red-5mm.html
27. Adafruit Learn. "Arduino Lesson 13: DC Motors — Transistors." https://learn.adafruit.com/adafruit-arduino-lesson-13-dc-motors/transistors
28. Adafruit. "N-channel power MOSFET — 30V / 60A (IRLB8721)." https://www.adafruit.com/product/355
29. Adafruit Learn. "TB6612 H-Bridge Motor Driver Breakout." https://learn.adafruit.com/adafruit-tb6612-h-bridge-dc-stepper-motor-driver-breakout/overview
30. Adafruit Learn. "Li-Ion and LiPoly Batteries." https://learn.adafruit.com/li-ion-and-lipoly-batteries/conclusion
31. STMicroelectronics. L298 dual full-bridge driver. https://www.st.com/en/motor-drivers/l298.html
32. Texas Instruments. DRV8833. https://www.ti.com/product/DRV8833
33. Wikipedia. "Flyback diode." https://en.wikipedia.org/wiki/Flyback_diode
34. University of Illinois ECE 110. "Electronics Lab Safety." https://courses.engr.illinois.edu/ece110/content/labs/archive/Skill_Safety_ElectronicsLab.pdf

Images

35. Shoot stufz. "Arduino Uno R3 development board 03." Wikimedia Commons, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Arduino_Uno_R3_development_board_03.jpg
36. Suyash Dwivedi. "25mm light-dependent resistor (LDR) (2)." Wikimedia Commons, CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:25mm_light-dependent_resistor_(LDR)_(2).jpg

Starter references for the other units (moved from the earlier draft)

37. Cisco. "What Is Computer Networking?" https://www.cisco.com/site/us/en/learn/topics/networking/what-is-computer-networking.html

38. Brooks, R. A. "A robust layered control system for a mobile robot." *IEEE Journal of Robotics and Automation*, 2(1), 14–23 (1986). https://ieeexplore.ieee.org/document/1087032
39. Oikawa, K., Tsuchiya, T. and Okubo, S. "Object-Oriented Design of Subsumption Architecture." *Journal of the Robotics Society of Japan*, 23(6), 697–705 (2005). https://www.jstage.jst.go.jp/article/jrsj1983/23/6/23_6_697/_article

Hex, buses, clock and architecture (A3, A5, A6 additions)

40. Arduino. Integer constants (`0x`, `0b`). https://docs.arduino.cc/language-reference/en/variables/constants/integerConstants/
41. Save My Exams. "Hexadecimal" and "Uses of hexadecimal" (Cambridge IGCSE). https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/1-data-representation/1-1-number-systems/hexadecimal *(secondary source)*
42. Save My Exams. "Von Neumann Architecture" (Cambridge IGCSE). https://savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/von-neumann-architecture *(secondary source)*
43. Seneca Learning. "The fetch-decode-execute cycle" (Cambridge IGCSE). https://senecalearning.com/en-GB/revision-notes/igcse/computer-science/cambridge-international/1-3-4-the-fetch-decode-execute-cycle *(secondary source)*
44. Toshiba. "Types of bus lines" (microcontroller e-learning). https://toshiba.semicon-storage.com/ap-en/semiconductor/knowledge/e-learning/micro-intro/chapter2/types-of-bus-lines.html
45. University of Cambridge, Department of Engineering. Microprocessor notes — address bus. https://www-mdp.eng.cam.ac.uk/web/library/enginfo/mdp_micro/lecture1/lecture1-3-4.html
46. TutorChase. AQA A-level 16.1.4 "Buses." https://www.tutorchase.com/notes/aqa-a-level/computer-science/16-1-4-buses *(secondary source)*
47. Emory University CS255. Computer overview notes (clock speed). https://cs.emory.edu/~cheung/Courses/255/Syllabus/HardCopy/01-overview-computer/03.pdf
48. Microchip. AVR CPU core — overview and instruction execution timing (megaAVR online docs). https://onlinedocs.microchip.com/oxy/GUID-74F8229E-4C43-4FA0-BE7D-1AA303C6F8A4-en-US-6/GUID-00885335-14FC-4AFF-9D60-0980181CAE0B.html
49. Wikipedia. "ATmega328." https://en.wikipedia.org/wiki/ATmega328 *(secondary source)*

Serial communication, capacitors, datasheets and sustainability (Part C)

50. SparkFun Learn. "Serial Communication." https://learn.sparkfun.com/tutorials/serial-communication/all
51. SparkFun Learn. "I2C." https://learn.sparkfun.com/tutorials/i2c/all
52. SparkFun Learn. "Serial Peripheral Interface (SPI)." https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all
53. Arduino. `Serial`, `Serial.begin()`, `Wire`, `Wire.setClock()`, `SPI` and `SPISettings` references (source: https://github.com/arduino/reference-en). https://docs.arduino.cc/language-reference/
54. Adafruit Learn. "Adafruit BME280 Humidity + Barometric Pressure + Temperature Sensor Breakout — Pinouts." https://learn.adafruit.com/adafruit-bme280-humidity-barometric-pressure-temperature-sensor-breakout/pinouts
55. RIOT-OS. BH1750FVI driver documentation (I^2C addresses). https://doc.riot-os.org/group__drivers__bh1750fvi.html *(secondary source)*
56. pySerial documentation. "Short introduction." https://pyserial.readthedocs.io/en/latest/shortintro.html
57. SparkFun Learn. "Capacitors." https://learn.sparkfun.com/tutorials/capacitors/all

58. Microchip. "Decoupling capacitors" (hardware design guidelines, online docs). https://onlinedocs.microchip.com/oxy/GUID-D4CAD149-CB72-498B-B8E7-7E8255593463-en-US-7/GUID-832C2C5F-69CC-40FC-8C9B-C00BDCF7A081.html
59. Texas Instruments. THS4281 datasheet — power supply recommendations. https://ti.com/document-viewer/THS4281/datasheet/power_supply_recommendations
60. Microchip. AVR analog input circuitry (ADC source impedance). https://onlinedocs.microchip.com/oxy/GUID-6178B82B-1DE0-484B-ACA5-153F78A78E41-en-US-1/GUID-085B188C-E869-4105-8411-EB13F0E66D65.html
61. Adafruit Learn. "Arduino Lesson 14: Servo Motors — If the servo misbehaves." https://learn.adafruit.com/adafruit-arduino-lesson-14-servo-motors/if-the-servo-misbehaves
62. Rapid Electronics / TruOpto. GL5528 photoresistor datasheet. https://static.rapidonline.com/pdf/58-0134_v1.pdf
63. Kingbright. WP7113ID 5 mm red LED datasheet. https://www.kingbrightusa.com/images/catalog/SPEC/WP7113ID.pdf
64. UNITAR. "Global E-waste Monitor 2024: Electronic waste rising five times faster than documented e-waste recycling" (press release). https://www.unitar.org/about/news-stories/press/global-e-waste-monitor-2024-electronic-waste-rising-five-times-faster-documented-e-waste-recycling
65. European Commission. "RoHS Directive." https://environment.ec.europa.eu/topics/waste-and-recycling/rohs-directive_en
66. US EPA. "Used household batteries." https://www.epa.gov/recycle/used-household-batteries

Status note

This file is **V1.1, a draft**. Part A and sections B3, B4, B7 (resolution/smoothing/hysteresis), B9 and B10 are new in V1.1. Sources were gathered with AI-assisted research (Claude, 22 Oct 2026); the research notes are in research_notes/ and a summary report is in reports/. The hexadecimal, buses/clock and von Neumann vs Harvard subsections (A3, A5, A6) and Part C were added on 23 Oct 2026, also from AI-assisted research; their notes are in research_notes/Hardware unit gap topics/ and the report, with a checklist of every secondary-source item, is reports/Hardware unit gap topics.md. The team still needs to check the content, build and test the circuit, and record this AI use in the project log before marking any section *Checked*.